



SAMARPAN INSTITUTE OF COMPUTER SCIENCE

Department of Computer Applications · Project Documentation

SARVOTAM CONSTRUCTION

End-to-End Construction ERP · Technical System Report

Project	Sarvotam Construction — Cloud ERP Platform
Tech Stack	React 18 · TypeScript · Vite · Supabase (PostgreSQL + Auth + Storage + Realtime)
Architecture	SPA + Serverless Backend (BaaS)
Database	PostgreSQL 15 (Supabase hosted)
Deployment	Vite production build, static hosting
Report Date	08 October 2026
Submitted By	Final-Year Project Team

Table of Contents

1. Abstract & Project Overview
2. Technology Stack
3. System Architecture
4. Database Design (Schema & Tables)
5. Authentication & Role-Based Access Control
6. Module-wise Functional Breakdown
7. Core Application Flows (with Flowcharts)
8. Frontend Structure & State Management
9. Security — RLS, Policies & Data Integrity
10. Realtime Sync & Offline Resilience
11. Error Handling, Logging & UX
12. Conclusion & Future Scope

1. Abstract & Project Overview

Sarvotam Construction is a full-stack web-based Enterprise Resource Planning (ERP) platform purpose-built for small-to-mid-size construction firms. The system digitises the complete operational lifecycle of a construction business — from client onboarding and project estimation to worker attendance, material stock, vehicle logs, income/expense bookkeeping, site-progress photos, and owner financial reporting — all within a single, mobile-friendly portal.

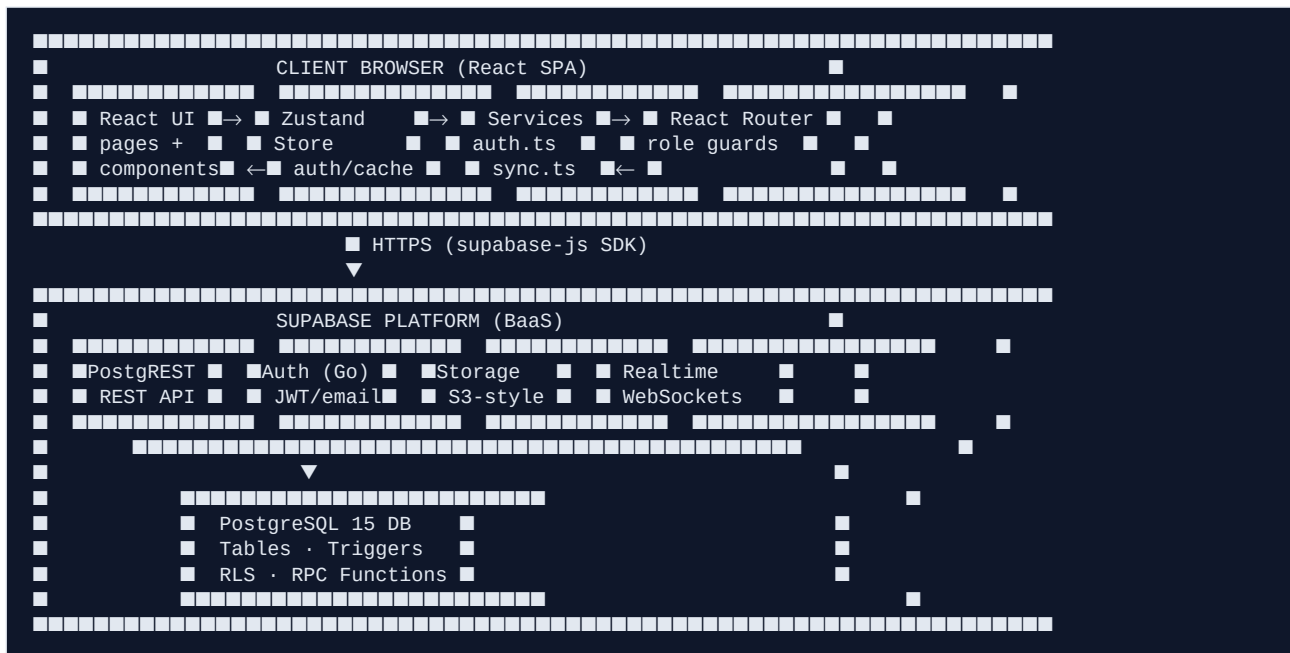
Problem Statement Traditional construction firms rely on paper registers, WhatsApp photos and scattered Excel sheets, leading to lost attendance slips, unverified material bills, delayed client payments and no real-time view of project health.	Our Solution A cloud-hosted, multi-role web app with live database sync. Owners, staff, supervisors, workers and clients each get a role-tailored dashboard backed by the same central database.
Key Outcomes Single source of truth for all project data, automated attendance reconciliation, photo proof-of-work, role-based access control, pending-approval signup, instant cross-device sync and professional financial reports.	Why Supabase (BaaS)? Provides managed PostgreSQL, Row-Level Security, object storage, realtime subscriptions and email-based auth out-of-the-box — eliminating the need to build & host a separate backend server.

2. Technology Stack

Layer	Technology	Purpose
UI	React 18 + TypeScript	Type-safe component-driven frontend with JSX for declarative UI
Build	Vite 8	Lightning-fast dev server + production bundling with HMR
Styling	Tailwind CSS	Rapid responsive utility-first styling
Charts	Recharts	Revenue, expense and progress dashboards
State	Zustand	Lightweight global store (auth, data cache, UI flags)
Routing	React Router v6	Protected routes per user role with nested layouts
Backend	Supabase (PostgreSQL 15)	Relational database + auto-generated REST API via PostgREST
Auth	Supabase Auth (JWT)	Email/password signup, sessions, password reset
Storage	Supabase Storage	Site-photo uploads, signed URLs, public read
Realtime	Supabase Realtime (WAL)	Broadcast DB changes to all devices in ~1 second
Security	PostgreSQL RLS	Row-level policies enforce role visibility server-side
Testing	Vitest + RTL	Smoke tests for each major page render

3. System Architecture

The application follows a **thick-client / serverless-backend** architecture. The React SPA talks directly to Supabase over HTTPS using an auto-generated REST API. No custom Node/Express server is maintained — all business rules are enforced inside PostgreSQL via functions, triggers and RLS policies.



4. Database Design

The schema is fully normalised to **3NF** and uses foreign keys, unique constraints and check constraints for referential integrity. Every table carries `created_at` timestamps and a UUID v4 primary key.

4.1 Tables & Relationships

Table	Purpose	Key Columns	Relations
profiles	Extends auth.users with role + approval status	id, name, email, phone, role, status, linked_id	1:1 with auth.users; linked_id → clients/workers
join_codes	One-time invite codes for staff/supervisor onboarding	code, role, used, created_by	created_by → profiles.id
clients	Customer master	name, phone, email, address, gst, type	Referenced by projects, income, estimates
projects	Active / planning / completed construction sites	name, client_id, budget, dates, status, progress	FK → clients
workers	Labour register with daily rate and bank details	name, skill, phone, rate, aadhaar, bank, ifsc	Referenced by attendance
attendance	Daily muster roll with advance tracking	worker_id, project_id, date, status, advance	FK → workers, projects; UNIQUE(worker_id, date)
materials	Incoming material register	name, vendor, project_id, qty, rate, paid	FK → projects
income	Payments received from clients	client_id, project_id, amount, mode, reference	FK → clients, projects
expenses	Operational spends	category, project_id, amount, mode	FK → projects
vehicles	Machinery register (JCB, truck, crane, ...)	name, number, type, owner_type, rate_per_hour	Referenced by vehicle_logs
vehicle_logs	Daily hours + diesel per machine	vehicle_id, project_id, hours, diesel_litres	FK → vehicles, projects
estimates	Client quotations (GST inclusive)	estimate_no, client_id, items JSONB, total, status	FK → clients, projects
site_photos	Proof-of-work images (path in Storage)	project_id, title, file_path, uploader_id	FK → projects, profiles

Design note: `estimates.items` is stored as `jsonb` because estimate line items are always accessed with the parent estimate and never queried independently. This avoids a join table while keeping the rest of the schema relational.

5. Authentication & Role-Based Access Control

5.1 Five User Roles

5.2 Signup & Approval Flow

Self-registration is allowed for **clients** and **workers**. Staff/supervisors need a one-time *join code* from the owner. The very first account is auto-promoted to admin/active so the system can bootstrap itself without manual SQL.

```
User visits /signup
  ▼
Fills name, email, password, role
  ▼
  ■■■ Role = admin/staff/supervisor?
    ▼
    ■ Join code validated (join_codes table)
      ▼
supabase.auth.signUp(email, password)
  ▼
DB TRIGGER handle_new_user() fires
  ▼
  ■■■ Fresh install? → role=admin, status=active
  ■■■ Valid join code? → requested role, status=pending
  ■■■ Client/worker? → role=client/worker, status=pending
    ▼
Frontend retries fetchMyProfile 6× (trigger latency)
  ■ (fallback manual INSERT if trigger fails)
    ▼
Sign out restricted session → navigate to /pending
  ▼
Toast: "Admin ko request chali gayi hai"
  ▼
Owner opens /admin/users → Approve / Reject
  ▼
User logs in → routed to role home (/client/dashboard, ...)
```

6. Module-wise Functional Breakdown

6.1 Admin / Staff Modules (/admin/*)

Module	Features
Dashboard	KPI tiles, revenue vs expense chart, project progress bars, recent activity
Clients	Add/edit clients (Individual/Company/Government), GST, contact, search/filter
Projects	Create project, assign client, set budget/timeline, status lifecycle, % progress
Workers	Worker register with skill, daily rate, Aadhaar, bank/IFSC; activate/deactivate
Attendance	Daily muster by project, Present/Half-Day/Absent/Holiday, advance cash; unique constraint
Materials	Vendor, qty, unit, rate, bill number, paid status; auto stock remaining
Income	Client payments with mode (Cash/Cheque/NEFT/UPI/RTGS/Online) and reference
Expenses	Categorised spends (Fuel, Labour, Equipment, Transport, Food, Safety, Tax, ...)
Vehicles	Machinery register, rate-per-hour; daily log for hours + diesel + operator
Estimates	Line-by-line quotations, auto subtotal/GST/total, Draft/Sent/Approved/Rejected
Photos	Shared board of site-progress photos with uploader name & timestamp
Reports	Date-range P&L, worker wage summary, project-wise cost break-up
Users (admin only)	Pending approvals, approve/reject signups, assign join codes, link users
Settings (admin only)	Company profile, GST, join codes, data export

6.2 Client Portal (/client/*)

- **Dashboard** — Active projects summary, total paid vs outstanding, latest photos.
- **Projects** — Timeline, progress bar, description of assigned sites.
- **Estimates** — View quotations sent, approve/reject digitally.
- **Payments** — Payment history and outstanding ledger.
- **Photos** — Real-time feed of site-progress imagery.
- **Profile** — Edit own contact details.

6.3 Worker Portal (/worker/*)

- **Dashboard** — This month's attendance count, earnings, advance deducted, net payable.
- **Attendance** — Calendar view of own present/absent/half-day marks.
- **Earnings** — Month-wise wage slip: days × rate – advance.
- **Profile** — Personal + bank details (edits need owner approval).

6.4 Supervisor Portal

Subset of admin views focused on field operations: mark attendance, add material entries, upload photos, view assigned projects. Finance modules are hidden by both React Router guards *and* RLS policies.

7. Core Application Flows

7.1 Login & Session Bootstrap

```

App loads → main.tsx mounts
  ■
  ▼
supabase.auth.getSession() ← JWT from localStorage
  ■
  ■■■ no session → authProfile = null, render public routes
  ■
  ■■■ has session → SELECT * FROM profiles WHERE id = auth.uid()
    ■
    ■■■ no profile → signOut + show signup/login
    ■■■ pending → redirect to /pending
    ■■■ active → set profile, startRealtime()
      ■
      ▼
      pullAll() – parallel fetch all tables
      ■
      ▼
      Navigate to //dashboard

```

7.2 Attendance Marking → Payroll Calculation

```

Supervisor opens /admin/attendance
  ■
  ▼
Select project + date → workers list loads
  ■
  ▼
Marks Present/Half/Absent per worker, enters advance ■
  ■
  ▼
INSERT INTO attendance (worker_id, project_id, date, status, advance)
  ■ · UNIQUE(worker_id, date) prevents duplicates
  ■ · RLS: only manager/supervisor can write
  ▼
Realtime pushes new row to all online devices in ~500ms
  ■
  ▼
Worker sees updated attendance on /worker/attendance
  ■
  ▼
End of month – wage engine:
  Present → +1 day × rate
  Half Day → +0.5 day × rate
  Holiday/Abs → 0
  Sum advance → subtract from gross → net payable

```

7.3 Photo Upload Pipeline

```

Any active user picks image on Photos page
  ■
  ▼
Client resizes/compresses (max 1920px, JPEG 85%)
  ■
  ▼
supabase.storage.from('site-photos').upload(path, file)
  ■
  ▼
getPublicUrl() → public CDN URL
  ■
  ▼
INSERT INTO site_photos (title, project_id, file_path, file_url, uploader_*)
  ■
  ▼
Realtime broadcast → PhotosBoard subscribes → image appears instantly

```


7.4 Financial Reporting Engine

All reports are computed **client-side** from cached table data using reduce/filter pipelines. This keeps the DB schema simple (no heavy SQL views) while producing instant reports on cached data. Example — monthly Profit & Loss:

```
const profit = income.filter(i => monthOf(i.date) === month)
  .reduce((s,i) => s + i.amount, 0)
- expenses.filter(e => monthOf(e.date) === month)
  .reduce((s,e) => s + e.amount, 0)
- attendance.filter(a => monthOf(a.date) === month
  && a.status !== 'Absent')
  .reduce((s,a) => {
    const w = workers.find(x => x.id === a.worker_id);
    const days = a.status === 'Half Day' ? 0.5 : 1;
    return s + (w?.rate ?? 0) * days - a.advance;
  }, 0);
```

8. Frontend Structure & State Management

```

src/
  App.tsx           ← router, role guards (RequireRole / PublicOnly)
  main.tsx          ← ReactDOM mount, Zustand bootstrap
  index.css         ← Tailwind layers + custom theme tokens
  types.ts          ← every shared TypeScript interface (single source)
  assets/           ← static images
  components/
    layout.tsx      ← Sidebar, topbar, role-aware navigation
    ui.tsx          ← Button, Input, Card, Modal, Select primitives
    charts.tsx      ← Recharts wrappers (RevenueChart, ProgressChart)
    ErrorBoundary.tsx ← Catches render errors, friendly recovery screen
  pages/
    public/         ← Landing, Signup/Login, Pending, Forgot/Reset
    admin/          ← Dashboard, Crud grids, Finance, Site, System
    client/          ← ClientPages (nested tabs)
    worker/         ← WorkerPages
    supervisor/     ← SupervisorPages
    shared/         ← PhotosBoard (used by all roles)
  services/
    supabaseClient.ts ← singleton createClient(SUPABASE_URL, KEY)
    auth.ts         ← cloudSignUp, cloudLogin, cloudSignOut, reset
    sync.ts         ← pullAll, saveToCloud, uploadPhoto, Realtime
    adapters.ts     ← DB row ↔ UI model mapping helpers
  store/
    appStore.ts     ← Zustand store (global state + async actions)
  lib/             ← small utilities (date, currency, CSV export)

```

8.1 Zustand Store Shape

Slice	Contents
auth	authProfile, authChecked, bootstrapAuth(), login(), signup(), signout()
data	clients[], projects[], workers[], attendance[], materials[], income[], expenses[], vehicles[], vehicleLogs[], estimates[], photos[], notifications[]
ui	busy, toast, sidebar collapse state
actions	refreshFromCloud(), save(), remove() — all funnel through sync.ts

8.2 Routing & Guards

- **PublicOnly** — redirects logged-in users away from /login and /signup.
- **RequireRole roles=[...]** — checks authProfile.role + active status; else /pending or /login.
- **homeFor(role)** — maps role to landing path (/admin/dashboard, /client/dashboard, ...).
- Nested routes share a **PortalLayout** (sidebar + header + tab bar) for role-tailored navigation.

9. Security — RLS, Policies & Data Integrity

Supabase exposes PostgREST publicly, so **all security is enforced inside the database**. React-side checks are cosmetic only.

9.1 Helper PL/pgSQL Functions

Function	Returns	Purpose
app_role()	text	Reads role from profiles for current JWT uid

app_is_active()	boolean	True only when status = active
app_is_manager()	boolean	True for admin and staff

9.2 Policy Matrix

Operation	Allowed Roles
SELECT	Any authenticated active user. Clients see only linked rows (per-table policies).
INSERT / UPDATE / DELETE	Managers (admin+staff); supervisors may additionally write attendance, materials, site_photos.
Own profile	User may read own row; may update name/phone but NOT role/status (trigger-guarded).
Join codes	Managers only — never visible to clients/workers.
Site photos	Any active user may insert; owner deletes own; managers delete any; reads are public (CDN).

Critical safety net: even if a malicious user edits React state or crafts a raw HTTP request to PostgREST, PostgreSQL will reject any operation outside their role. This is the defence-in-depth guarantee of Supabase BaaS.

9.3 Other Safeguards

- **Trigger-based profile creation** — every auth.users row gets a profiles row; no orphan sessions.
- **Unique constraints** — (worker_id, date) on attendance blocks double-marking even during offline collisions.
- **Check constraints** — role/status columns validated against allowed literal values.
- **Foreign keys** — ON DELETE RESTRICT prevents loss of linked financial records.
- **Error normalisation** — Supabase error objects sometimes serialise to {}; UI now stringifies & filters them to friendly messages.

10. Realtime Sync & Offline Resilience

After login, `startRealtime()` subscribes a single Postgres-changes channel covering all business tables. When any INSERT/UPDATE/DELETE lands, the WAL event is broadcast over WebSocket and merged into Zustand within ~500 ms — so the owner's dashboard updates while a supervisor is still marking attendance from the field.

10.1 Resilient Bootstrap

After signup the Postgres trigger can take 100–400 ms to commit the new profile row. The frontend retries `fetchMyProfile()` up to **6 times with 600 ms back-off**, and as a final defensive measure performs a manual INSERT into profiles if the trigger still hasn't fired. This guarantees the admin always sees the pending request in `/admin/users`.

11. Error Handling, Logging & UX

- **ErrorBoundary** at the React root catches render exceptions and shows a friendly recovery screen with Refresh / Go Home.
- **Sanitised error messages** — every catch block normalises errors to strings, guarding against empty objects {}, [object Object], undefined messages and timeouts.
- **Bilingual toasts** — Success messages use Hindi (e.g. "Admin ko request chali gayi hai") for field workers while admin UI stays English.
- **Loading flags (busy)** — every submit button disables with a spinner via `finally`, preventing double-submits and stuck states.
- **Pending-approval screen** — `/pending` clearly tells new users they are waiting on owner approval.
- **Responsive layout** — Sidebar collapses on mobile; tables become card lists below 768 px via Tailwind breakpoints.

12. Conclusion & Future Scope

Sarvotam Construction delivers a production-ready ERP that replaces paper-based construction workflows with a unified, cloud-native platform. By combining React + TypeScript with Supabase's managed Postgres + RLS, the system achieves enterprise-grade access control, realtime multi-device sync and media storage — all without the operational overhead of a dedicated backend server.

12.1 Future Enhancements

— End of Report —

Prepared as part of the BCA / B.Sc. Computer Science curriculum at Samarpan Institute of Computer Science.